

Clase de Consultas

1er parcial

Lautaro Lasorsa - Tobias Carreira Munich - Ezequiel Companeeetz

Departamento de Computación
Facultad de Ciencias Exactas y Naturales
Universidad de Buenos Aires

2^{do} Cuatrimestre de 2025

1 Ejercicios Múltiple Choice

2 Máximin D&C

3 Viaje a Mar del Plata

4 RutaMinima

5 Pila Cauta

Pregunta: Teorema Maestro

Si se tiene una recurrencia de la forma:

$$T(n) = aT\left(\frac{n}{b}\right) + f(n)$$

¿Cuáles de los siguientes valores para a , b y $f(n)$ aseguran que la complejidad final sea $\Theta(\sqrt{n} \log n)$?

- $a = 4, b = 16, f(n) = n^{1/2}$
- $a = 2, b = 2, f(n) = n^{1/2} \log n$
- $a = 2, b = 8, f(n) = n^{1/2} \log n$
- $a = 4, b = 2, f(n) = 1$
- $a = 3, b = 9, f(n) = n^{1/4}$

Solución: Teorema Maestro

Respuesta correcta:

$$(1) a = 4, b = 16, f(n) = n^{1/2} \quad y \quad (3) a = 2, b = 8, f(n) = n^{1/2} \log n$$

Explicación

- Para $a = 4, b = 16$: $\log_b a = \log_{16} 4 = 1/2 \Rightarrow n^{\log_b a} = n^{1/2}$. Caso 2 del Teorema Maestro $\Rightarrow T(n) = \Theta(n^{1/2} \log n)$.
- Para $a = 2, b = 8$: $\log_8 2 = 1/3 \Rightarrow n^{\log_b a} = n^{1/3}$, pero $f(n) = n^{1/2} \log n$ es mayor (polinómicamente). Caso 3: $T(n) = \Theta(f(n)) = \Theta(\sqrt{n} \log n)$.
- Las demás opciones no cumplen la condición:
 - $(a = 2, b = 2)$ da $n^{\log_2 2} = n$ (mayor que $\sqrt{n} \log n$).
 - $(a = 4, b = 2)$ da $n^{\log_2 4} = n^2$ (cuadrático).
 - $(a = 3, b = 9)$ da $n^{\log_9 3} = n^{1/2}$ pero $f(n) = n^{1/4}$ es menor $\Rightarrow$ caso 1 $\Rightarrow \Theta(n^{1/2})$.

Pregunta: Backtracking

¿Cuáles de estas afirmaciones son verdaderas para soluciones basadas en backtracking?

- Evalúan todas las soluciones candidatas posibles.
- Anticipan que ciertos subconjuntos no son factibles y los descartan.
- Explotan superposición de subproblemas.
- Siempre mejoran la complejidad de fuerza bruta.
- Siempre tienen complejidad exponencial.
- Pueden usarse en problemas sin algoritmo polinomial conocido.

Solución: Backtracking

Correctas: 1, 2, 6

Explicación

- (1) Verdadero si no hay podas: explora todo el espacio de soluciones.
- (2) Verdadero: usa podas para descartar ramas no factibles.
- (3) Falso: superposición es propia de DP, backtracking no guarda resultados.
- (4) Falso: puede tener el mismo costo que fuerza bruta si no hay podas efectivas.
- (5) Falso: en algunos problemas es lineal (p.ej. DFS para path existence).
- (6) Verdadero: se usan en NP-completos como knapsack (0/1).

Pregunta: Complejidad espacial DP

La complejidad espacial de la memorización de una *DP* con N estados y M transiciones entre estados, implementada en forma top-down recursiva, es:

- $\Theta(M)$
- $\Theta(N + M)$
- $\Theta(N)$
- $\Theta(NM)$

Solución: Complejidad espacial DP

Respuesta correcta: $\Theta(N)$

Explicación

- Solo se guarda un valor por cada estado (N en total).
- Las transiciones M no ocupan espacio adicional en la memo.
- La pila recursiva agrega $O(\text{profundidad}) \leq O(N)$ en el peor caso.
- Por eso el uso de memoria es proporcional al número de estados.

Pregunta: Estrategias Greedy

¿Cuál(es) de las siguientes afirmaciones describe(n) a las estrategias greedy?

- Siempre garantizan solución óptima.
- Exploran exactamente una rama del árbol de backtracking definido por la recursión.
- Para considerarlas algoritmos correctos, se necesita demostración de optimalidad.
- Siempre son estrictamente mejores que cualquier otro algoritmo.

Solución: Estrategias Greedy

Correctas: 2 y 3

Explicación

- (1) Falsa: hay problemas donde greedy falla (ej. cambio de monedas no canónico).
- (2) Verdadera: un greedy recorre una sola rama de decisiones.
- (3) Verdadera: se requiere prueba formal de optimalidad.
- (4) Falsa: no siempre es mejor en complejidad o resultado que otras técnicas.

Máximo y mínimo con menos comparaciones

Enunciado

Se tiene un array A de n elementos (por simplicidad, n potencia de 2) del cual se quiere obtener su máximo y su mínimo. La solución secuencial compara cada elemento contra el máximo y el mínimo actuales. Si la comparación es costosa, queremos una alternativa que **minimice la cantidad de comparaciones** sin empeorar el tiempo asintótico.

Proponer y justificar un algoritmo de D & C.

Idea: Divide & Conquer (minimizando comparaciones)

- **Dividir:** partir el arreglo en dos mitades.
- **Resolver:** obtener (mín, máx) de cada mitad recursivamente.
- **Combinar:** el mínimo global es el menor de los mínimos; el máximo global es el mayor de los máximos.

Caso base

- 1 elemento $\Rightarrow$ ese valor es mínimo y máximo.

Divide & Conquer para Mínimo y Máximo

Pseudocódigo

ObtenerMaxMin(A , inicio, fin):

- Si $inicio = fin$, devolver ($A[inicio]$, $A[inicio]$) (caso base: un solo elemento)
- Sea $medio \leftarrow \lfloor (inicio + fin)/2 \rfloor$
- Sea $(min_izq, max_izq) \leftarrow \text{ObtenerMaxMin}(A, inicio, medio)$
- Sea $(min_der, max_der) \leftarrow \text{ObtenerMaxMin}(A, medio + 1, fin)$
- $min_global \leftarrow \min(min_izq, min_der)$
- $max_global \leftarrow \max(max_izq, max_der)$
- devolver (min_global, max_global)

Analizando la complejidad

Objetivo

Queremos ver si nuestro algoritmo tiene la **misma complejidad** que encontrar el máximo y el mínimo secuencialmente.

Analizando la complejidad

Objetivo

Queremos ver si nuestro algoritmo tiene la **misma complejidad** que encontrar el máximo y el mínimo secuencialmente.

Recurrencia

La recurrencia es:

$$T(n) = 2 T\left(\frac{n}{2}\right) + O(1)$$

porque hacemos dos llamadas recursivas y combinamos en tiempo constante.

Analizando la complejidad

Objetivo

Queremos ver si nuestro algoritmo tiene la **misma complejidad** que encontrar el máximo y el mínimo secuencialmente.

Recurrencia

La recurrencia es:

$$T(n) = 2 T\left(\frac{n}{2}\right) + O(1)$$

porque hacemos dos llamadas recursivas y combinamos en tiempo constante.

- Aplicaremos el **Teorema Maestro** para resolver esta recurrencia.

Resolviendo con el Teorema Maestro

Resultado

Caso 2 del Teorema Maestro: $a = 2$, $b = 2$, $f(n) = O(1)$.

$$n^{\log_b a} = n^{\log_2 2} = n \Rightarrow T(n) = \Theta(n)$$

Por lo tanto, **nuestro algoritmo es lineal**, igual que la solución secuencial.

Resolviendo con el Teorema Maestro

Resultado

Caso 2 del Teorema Maestro: $a = 2$, $b = 2$, $f(n) = O(1)$.

$$n^{\log_b a} = n^{\log_2 2} = n \Rightarrow T(n) = \Theta(n)$$

Por lo tanto, **nuestro algoritmo es lineal**, igual que la solución secuencial.

Cantidad de comparaciones

- **Secuencial:** $2(n - 1)$ comparaciones.
- **D&C:** $\approx \frac{3}{2}n - 2$ (para n potencia de 2), que es menor que $2n - 2$. (esto surge de calcular con exactitud la serie geométrica $(n/2 + n/4 + \dots)$)

Ejercicio 2 (Viaje a Mar del Plata)

Planteo

Tomás quiere viajar de Buenos Aires a Mar del Plata en su flamante Renault 12. Como está preocupado por la autonomía de su vehículo, se tomó el tiempo de anotar las distintas estaciones de servicio que se encuentran en el camino.

Modelo

Modeló el camino como un segmento de 0 a M , donde Buenos Aires está en el kilómetro 0, Mar del Plata en M , y las estaciones de servicio están ubicadas en:

$$0 = x_1 \leq x_2 \leq \dots \leq x_n \leq M.$$

Consigna

Objetivo

Minimizar la **cantidad de paradas** para cargar nafta. El auto puede hacer hasta C km con el tanque lleno, y al comenzar el viaje el tanque está vacío.

- Proponer un algoritmo *greedy* que indique la **cantidad mínima de paradas** y devuelva el **conjunto de estaciones** donde detenerse. **Probar su correctitud.**
- Dar una implementación de **complejidad temporal** $O(n)$.

Algoritmo Greedy propuesto

Idea general

La estrategia greedy es simple pero efectiva:

siempre avanzar hasta la estación más lejana posible dentro del alcance C km.

Descripción intuitiva

Comenzando desde Buenos Aires (o desde la última estación donde cargamos combustible), miramos todas las estaciones dentro de C kilómetros y elegimos la más lejana. Repetimos hasta llegar a Mar del Plata.

Pseudocódigo ($O(n)$)

Versión en pseudocódigo

Algoritmo: MinParadas($x[]$, n , C , M)

Entrada: $x[1..n]$ = posiciones de estaciones ($x[0] = 0$ implícito)

C = alcance máximo con tanque lleno

M = distancia total a Mar del Plata

Salida: Conjunto de estaciones donde parar

```
1.  posicion_actual <- 0
2.  estaciones <- {x[1]}  // Empezamos cargando en Buenos Aires
3.  idx <- 1

5.  Mientras posicion_actual + C < M hacer:
6.      ultima_alcanzable <- idx - 1

8.      // Buscar la estación más lejana alcanzable
```

Pseudocódigo ($O(n)$)

Versión en pseudocódigo

```
9.      Mientras idx <= n Y x[idx] <= posicion_actual + C hacer:
10.         ultima_alcanzable <- idx
11.         idx <- idx + 1
12.      Fin Mientras

14.      Si ultima_alcanzable == idx - 1 entonces:
15.         // No hay estación alcanzable
16.         Retornar "No hay solución"
17.      Fin Si

19.      estaciones <- estaciones U {x[ultima_alcanzable]}
20.      posicion_actual <- x[ultima_alcanzable]
21.  Fin Mientras

23.  Retornar estaciones
```

Ejemplo de ejecución

Datos

- Distancia total $M = 400$ km
- Alcance $C = 150$ km
- Estaciones en: $\{0, 80, 140, 200, 280, 350, 400\}$

Ejemplo de ejecución

Datos

- Distancia total $M = 400$ km
- Alcance $C = 150$ km
- Estaciones en: $\{0, 80, 140, 200, 280, 350, 400\}$

Iteración	Posición actual	Alcance hasta	Estación elegida
1	0	150	140 (más lejana ≤ 150)
2	140	290	280 (más lejana ≤ 290)
3	280	430	400 (destino alcanzable)

Solución

Paradas en km 0, 140 y 280. Total: **3 paradas**.

Correctitud — Propiedad Greedy y Subestructura Óptima

Lema 1 (Propiedad de elección Greedy)

Existe una solución óptima que incluye la primera elección del algoritmo greedy.

Demostración Sea g_1 la estación más lejana alcanzable desde el origen con alcance C . Para una solución óptima cualquiera $\mathcal{O} = \{o_1, \dots, o_k\}$:

Correctitud — Propiedad Greedy y Subestructura Óptima

Lema 1 (Propiedad de elección Greedy)

Existe una solución óptima que incluye la primera elección del algoritmo greedy.

Demostración Sea g_1 la estación más lejana alcanzable desde el origen con alcance C . Para una solución óptima cualquiera $\mathcal{O} = \{o_1, \dots, o_k\}$:

- Si $o_1 = g_1$, listo.
- Si $o_1 \neq g_1$, entonces $x_{o_1} < x_{g_1}$. Reemplazando o_1 por g_1 y manteniendo las restantes estaciones que están más allá de g_1 , la nueva solución es factible y no aumenta el número de paradas. ¿Cómo probamos que nuestra nueva solución es factible?

Correctitud — Propiedad Greedy y Subestructura Óptima

Lema 1 (Propiedad de elección Greedy)

Existe una solución óptima que incluye la primera elección del algoritmo greedy.

Demostración Sea g_1 la estación más lejana alcanzable desde el origen con alcance C . Para una solución óptima cualquiera $\mathcal{O} = \{o_1, \dots, o_k\}$:

- Si $o_1 = g_1$, listo.
- Si $o_1 \neq g_1$, entonces $x_{o_1} < x_{g_1}$. Reemplazando o_1 por g_1 y manteniendo las restantes estaciones que están más allá de g_1 , la nueva solución es factible y no aumenta el número de paradas. ¿Cómo probamos que nuestra nueva solución es factible? *Lo pueden ver en el handout.*

Así concluimos que cualquier solución óptima se puede modificar para que contenga a g_1 .

Lema 2: Subestructura Óptima

Enunciado

Lema 2 (Subestructura óptima). Si $\mathcal{O}$ es una solución óptima para llegar de 0 a M , y o_k es una estación en $\mathcal{O}$, entonces $\mathcal{O} \setminus \{o_1, \dots, o_k\}$ es una solución óptima para el subproblema de ir de x_{o_k} a M .

Lema 2: Subestructura Óptima

Enunciado

Lema 2 (Subestructura óptima). Si $\mathcal{O}$ es una solución óptima para llegar de 0 a M , y o_k es una estación en $\mathcal{O}$, entonces $\mathcal{O} \setminus \{o_1, \dots, o_k\}$ es una solución óptima para el subproblema de ir de x_{o_k} a M .

Demostración

Por contradicción. Supongamos que existe una solución S para ir de x_{o_k} a M con **menos paradas** que $\mathcal{O} \setminus \{o_1, \dots, o_k\}$.

Entonces:

$$\{o_1, \dots, o_k\} \cup S$$

sería una solución factible para el problema original con menos paradas que $\mathcal{O}$, contradiciendo la optimalidad de $\mathcal{O}$. □

Teorema: Correctitud del Algoritmo Greedy

Enunciado

Teorema. El algoritmo greedy produce una solución óptima.

Teorema: Correctitud del Algoritmo Greedy

Enunciado

Teorema. El algoritmo greedy produce una solución óptima.

Demostración (Construcción inductiva)

Sea $\mathcal{G} = \{g_1, g_2, \dots, g_k\}$ la solución producida por el algoritmo greedy, donde g_i denota la i -ésima estación elegida.

Probaremos por inducción que para cada $i \in \{1, \dots, k\}$ existe una solución óptima que contiene las primeras i elecciones greedy $\{g_1, \dots, g_i\}$.

Demostración del Teorema (1/2)

Caso base ($i = 1$): Por el Lema 1, existe una solución óptima $\mathcal{O}_1$ que contiene g_1 (la primera elección greedy).

Hipótesis inductiva: Supongamos que existe una solución óptima $\mathcal{O}_i$ que contiene $\{g_1, g_2, \dots, g_i\}$.

Paso inductivo: Queremos probar que existe una solución óptima que contiene también g_{i+1} .

- Por el Lema 2, si $\mathcal{O}_i$ es óptima para ir de 0 a M , entonces $\mathcal{O}'_i = \mathcal{O}_i \setminus \{g_1, \dots, g_i\}$ es óptima para ir de x_{g_i} a M .

Demostración del Teorema (1/2)

Caso base ($i = 1$): Por el Lema 1, existe una solución óptima $\mathcal{O}_1$ que contiene g_1 (la primera elección greedy).

Hipótesis inductiva: Supongamos que existe una solución óptima $\mathcal{O}_i$ que contiene $\{g_1, g_2, \dots, g_i\}$.

Paso inductivo: Queremos probar que existe una solución óptima que contiene también g_{i+1} .

- Por el Lema 2, si $\mathcal{O}_i$ es óptima para ir de 0 a M , entonces $\mathcal{O}'_i = \mathcal{O}_i \setminus \{g_1, \dots, g_i\}$ es óptima para ir de x_{g_i} a M .
- Aplicamos el mismo argumento del Lema 1 al subproblema: si la solución óptima $\mathcal{O}'_i$ no comienza con g_{i+1} , podemos reemplazar su primera estación por g_{i+1} sin aumentar el número de paradas y manteniendo la factibilidad.

Demostración del Teorema (1/2)

Caso base ($i = 1$): Por el Lema 1, existe una solución óptima $\mathcal{O}_1$ que contiene g_1 (la primera elección greedy).

Hipótesis inductiva: Supongamos que existe una solución óptima $\mathcal{O}_i$ que contiene $\{g_1, g_2, \dots, g_i\}$.

Paso inductivo: Queremos probar que existe una solución óptima que contiene también g_{i+1} .

- Por el Lema 2, si $\mathcal{O}_i$ es óptima para ir de 0 a M , entonces $\mathcal{O}'_i = \mathcal{O}_i \setminus \{g_1, \dots, g_i\}$ es óptima para ir de x_{g_i} a M .
- Aplicamos el mismo argumento del Lema 1 al subproblema: si la solución óptima $\mathcal{O}'_i$ no comienza con g_{i+1} , podemos reemplazar su primera estación por g_{i+1} sin aumentar el número de paradas y manteniendo la factibilidad.
- Quitando el primer elemento de $\mathcal{O}'_i$ obtenemos la solución óptima $\mathcal{S}$, para ir de $x_{g_{i+1}}$ a M .

Demostración del Teorema (2/2)

Construcción de la nueva solución

Definimos:

$$\mathcal{O}_{i+1} = \{g_1, \dots, g_i, g_{i+1}\} \cup S$$

donde S es una solución óptima para ir de $x_{g_{i+1}}$ a M .

Demostración del Teorema (2/2)

Construcción de la nueva solución

Definimos:

$$\mathcal{O}_{i+1} = \{g_1, \dots, g_i, g_{i+1}\} \cup \mathcal{S}$$

donde $\mathcal{S}$ es una solución óptima para ir de $x_{g_{i+1}}$ a M .

Conclusión

Por inducción, existe una solución óptima que contiene todas las elecciones greedy $\{g_1, \dots, g_k\}$.

Como el algoritmo greedy termina al alcanzar M , y ninguna solución puede usar menos paradas que una óptima, concluimos que:

$\mathcal{G}$ es una solución óptima. $\square$

Notas finales

Resumen

- Estrategia: elegir siempre la **estación más lejana** dentro del alcance C .
- Correctitud: propiedad de elección greedy, subestructura óptima y utilizar estas dos propiedades en nuestra inducción.
- Complejidad: $O(n)$.

Ejercicio 3 (RutaMinima)

Dada una matriz D de $n \times n$ números naturales, queremos encontrar una permutación π de $\{1, \dots, n\}$ que minimice $D_{\pi(n)\pi(1)} + \sum_{i=1}^{n-1} D_{\pi(i)\pi(i+1)}$. Por ejemplo, si

$$D = \begin{pmatrix} 0 & 1 & 10 \\ 10 & 0 & 3 \\ 2 & 22 & 0 \end{pmatrix},$$

entonces $\pi(i) = i$ es una solución óptima.

- 1 Diseñar un algoritmo de backtracking para resolver el problema, indicando claramente cómo se codifica una solución candidata, cuáles soluciones son válidas y qué valor tienen, qué es una solución parcial y cómo se extiende cada solución parcial.
- 2 Dibujar el árbol de recursión para el ejemplo.
- 3 Calcular la complejidad temporal del mismo.
- 4 Proponer una poda por optimalidad y mostrar que es correcta.

Ejercicio 3 (RutaMinima) - algoritmo

Diseñar un algoritmo de backtracking para resolver el problema, indicando claramente cómo se codifica una solución candidata, cuáles soluciones son válidas y qué valor tienen, qué es una solución parcial y cómo se extiende cada solución parcial.

Ejercicio 3 (RutaMinima) - algoritmo

Diseñar un algoritmo de backtracking para resolver el problema, indicando claramente cómo se codifica una solución candidata, cuáles soluciones son válidas y qué valor tienen, qué es una solución parcial y cómo se extiende cada solución parcial.

- Las soluciones candidatas son todas las permutaciones de los números del 1 al n : $\pi = (\pi_1, \dots, \pi_n)$ tal que $1 \leq \pi_i \leq n$ y todos los elementos de π son distintos.

Ejercicio 3 (RutaMinima) - algoritmo

Diseñar un algoritmo de backtracking para resolver el problema, indicando claramente cómo se codifica una solución candidata, cuáles soluciones son válidas y qué valor tienen, qué es una solución parcial y cómo se extiende cada solución parcial.

- Las soluciones candidatas son todas las permutaciones de los números del 1 al n : $\pi = (\pi_1, \dots, \pi_n)$ tal que $1 \leq \pi_i \leq n$ y todos los elementos de π son distintos.
- Las soluciones válidas son todas las candidatas.

Ejercicio 3 (RutaMinima) - algoritmo

Diseñar un algoritmo de backtracking para resolver el problema, indicando claramente cómo se codifica una solución candidata, cuáles soluciones son válidas y qué valor tienen, qué es una solución parcial y cómo se extiende cada solución parcial.

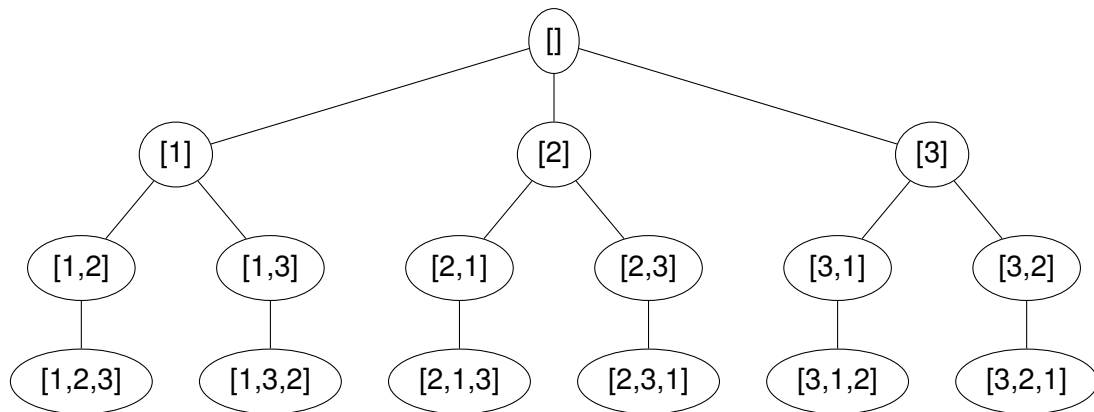
- Las soluciones candidatas son todas las permutaciones de los números del 1 al n : $\pi = (\pi_1, \dots, \pi_n)$ tal que $1 \leq \pi_i \leq n$ y todos los elementos de π son distintos.
- Las soluciones válidas son todas las candidatas.
- Las soluciones parciales son vectores $p = (p_1, \dots, p_i)$ con $1 \leq p_i \leq n$, todos los elementos de p son distintos y $0 \leq i \leq n$.

Ejercicio 3 (RutaMinima) - algoritmo

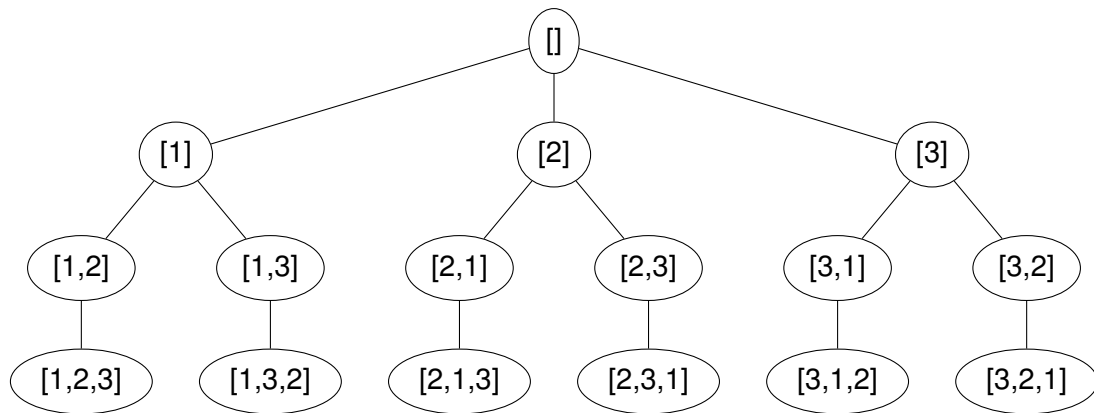
Diseñar un algoritmo de backtracking para resolver el problema, indicando claramente cómo se codifica una solución candidata, cuáles soluciones son válidas y qué valor tienen, qué es una solución parcial y cómo se extiende cada solución parcial.

- Las soluciones candidatas son todas las permutaciones de los números del 1 al n : $\pi = (\pi_1, \dots, \pi_n)$ tal que $1 \leq \pi_i \leq n$ y todos los elementos de π son distintos.
- Las soluciones válidas son todas las candidatas.
- Las soluciones parciales son vectores $p = (p_1, \dots, p_i)$ con $1 \leq p_i \leq n$, todos los elementos de p son distintos y $0 \leq i \leq n$.
- Las soluciones sucesoras de p son $p \oplus x$ con $1 \leq x \leq n$ y $x \notin p$.

Ejercicio 3 (RutaMinima) - árbol



Ejercicio 3 (RutaMinima) - árbol



Complejidad: $O(n!)$

Pila Cauta

Tenemos cajas numeradas de 1 a N , todas de iguales dimensiones. Queremos encontrar la máxima cantidad de cajas que pueden apilarse en una única pila cumpliendo que:

- sólo puede haber una caja apoyada directamente sobre otra;
- las cajas de la pila deben estar ordenadas crecientemente por número, de abajo para arriba;
- cada caja i tiene un peso w_i y un soporte s_i , y el peso total de las cajas que están arriba de otra no debe exceder el soporte de esa otra.

Si tenemos los pesos $w = [19, 7, 5, 6, 1]$ y los soportes $s = [15, 13, 7, 8, 2]$ (la caja 1 tiene peso 19 y soporte 15, la caja 2 tiene peso 7 y soporte 13, etc.), entonces la respuesta es 4. Por ejemplo, pueden apilarse de la forma 1-2-3-5 o 1-2-4-5 (donde la izquierda es más abajo), entre otras opciones.

Consigna

- 1 Pensar la idea de un algoritmo de *backtracking* (no hace falta escribirlo).
- 2 Escribir una formulación recursiva que sea la base de un algoritmo de PD. Explicar su semántica e indicar cuáles serían los parámetros para resolver el problema.
- 3 Diseñar un algoritmo de PD y dar su complejidad temporal y espacial auxiliar. Comparar cómo resultaría un enfoque *top-down* con uno *bottom-up*.
- 4 (Opcional) Formalizar el problema y demostrar que la función recursiva es correcta.

Idea de Backtracking (tentativa)

$$f(w, sec) = \begin{cases} evaluar(sec) & \text{si } w = [] \\ \max \begin{cases} f(w[1:], sec) \\ f(w[1:], sec + [w[0]]) \end{cases} & \text{si } w \neq [] \end{cases}$$

Donde

$$evaluar(secuencia) = \begin{cases} 0 & \text{si } secuencia \text{ no es válida} \\ |secuencia| & \text{si } secuencia \text{ es válida} \end{cases}$$

Idea de Backtracking (tentativa)

$$f(w, sec) = \begin{cases} evaluar(sec) & \text{si } w = [] \\ \max \begin{cases} f(w[1:], sec) \\ f(w[1:], sec + [w[0]]) \end{cases} & \text{si } w \neq [] \end{cases}$$

Donde

$$evaluar(secuencia) = \begin{cases} 0 & \text{si } secuencia \text{ no es válida} \\ |secuencia| & \text{si } secuencia \text{ es válida} \end{cases}$$

Respuesta:

Idea de Backtracking (tentativa)

$$f(w, sec) = \begin{cases} evaluar(sec) & \text{si } w = [] \\ \max \begin{cases} f(w[1:], sec) \\ f(w[1:], sec + [w[0]]) \end{cases} & \text{si } w \neq [] \end{cases}$$

Donde

$$evaluar(secuencia) = \begin{cases} 0 & \text{si } secuencia \text{ no es válida} \\ |secuencia| & \text{si } secuencia \text{ es válida} \end{cases}$$

Respuesta: $f(w, [])$

Complejidad:

Idea de Backtracking (tentativa)

$$f(w, sec) = \begin{cases} evaluar(sec) & \text{si } w = [] \\ \max \begin{cases} f(w[1:], sec) \\ f(w[1:], sec + [w[0]]) \end{cases} & \text{si } w \neq [] \end{cases}$$

Donde

$$evaluar(secuencia) = \begin{cases} 0 & \text{si } secuencia \text{ no es válida} \\ |secuencia| & \text{si } secuencia \text{ es válida} \end{cases}$$

Respuesta: $f(w, [])$

Complejidad: $O(2^N)$

Notar que evalúa literalmente cada posible subsecuencia de cajas.

Abstracción

Notar que:

Abstracción

Notar que:

- No me importa toda la secuencia de cajas que ya puse, sino sólo cuanto peso puedo poner arriba de ellas.

Abstracción

Notar que:

- No me importa toda la secuencia de cajas que ya puse, sino sólo cuanto peso puedo poner arriba de ellas.
- La secuencia de cajas está fija, por lo cuál no necesito pasar toda la secuencia como entrada, puedo utilizar únicamente un índice.

Abstracción

Notar que:

- No me importa toda la secuencia de cajas que ya puse, sino sólo cuanto peso puedo poner arriba de ellas.
- La secuencia de cajas está fija, por lo cuál no necesito pasar toda la secuencia como entrada, puedo utilizar únicamente un índice.

Con las observaciones anteriores, podemos definir la siguiente función: $f(i, p)$ = máxima cantidad de cajas que puedo apilar usando sólo las cajas $i, i + 1, \dots, N$ y teniendo p de peso disponible para poner arriba de la última caja puesta.

Formulación Recursiva

$$f(i, p) = \begin{cases} -\infty & \text{si } p < 0 \\ 0 & \text{si } i > N \\ \text{máx} \begin{cases} f(i+1, p) \\ 1 + f(i+1, \text{mín}(p - w_i, s_i)) \end{cases} & \text{si } i \leq N \end{cases}$$

Formulación Recursiva

$$f(i, p) = \begin{cases} -\infty & \text{si } p < 0 \\ 0 & \text{si } i > N \\ \max \left\{ \begin{array}{l} f(i+1, p) \\ 1 + f(i+1, \min(p - w_i, s_i)) \end{array} \right\} & \text{si } i \leq N \end{cases}$$

Sea

$$W = \min \left(\max_{1 \leq i \leq N} (w_i + s_i), \sum_{i=1}^N w_i \right)$$

·
Respuesta: $f(1, W)$

Formulación Recursiva

$$f(i, p) = \begin{cases} -\infty & \text{si } p < 0 \\ 0 & \text{si } i > N \\ \max \left\{ \begin{array}{l} f(i+1, p) \\ 1 + f(i+1, \min(p - w_i, s_i)) \end{array} \right\} & \text{si } i \leq N \end{cases}$$

Sea

$$W = \min \left(\max_{1 \leq i \leq N} (w_i + s_i), \sum_{i=1}^N w_i \right)$$

Respuesta: $f(1, W)$

Complejidad: $O(N * W)$

*Notar que la función planteada es lineal en W , pero W es un valor que no restringe el tamaño de la entrada. A esto se llama **complejidad pseudo-polynomial**.*

Implementaciones

Notar que en este caso podemos mejorar la complejidad espacial a $O(W)$ si utilizamos un enfoque iterativo.

Correctitud

Objetivo: Demostrar que la función recursiva es correcta.

Correctitud

Objetivo: Demostrar que la función recursiva es correcta.

Metodo: Inducción en los parametros de la función.

Correctitud

Objetivo: Demostrar que la función recursiva es correcta.

Metodo: Inducción en los parametros de la función.

Hipótesis inductiva: $H(i, p)$: La función $f(i, p)$ devuelve la máxima cantidad de cajas que puedo apilar usando sólo las cajas $i, i + 1, \dots, N$ y teniendo p de peso disponible para poner arriba de la última caja puesta.

Correctitud

Objetivo: Demostrar que la función recursiva es correcta.

Metodo: Inducción en los parametros de la función.

Hipótesis inductiva: $H(i, p)$: La función $f(i, p)$ devuelve la máxima cantidad de cajas que puedo apilar usando sólo las cajas $i, i + 1, \dots, N$ y teniendo p de peso disponible para poner arriba de la última caja puesta.

Paso inductivo: Supongamos que $H(i', p')$ es cierta para todo par (i', p') tales que $i' > i$. Queremos probar que $H(i, p)$ es cierta.

Correctitud (II)

Caso base: Si $p < 0$, entonces no podemos poner ninguna caja, por lo que $f(i, p) = -\infty$. Si $i > N$, entonces no quedan cajas para considerar, por lo que $f(i, p) = 0$. En ambos casos la invariante se cumple.

Correctitud (II)

Caso base: Si $p < 0$, entonces no podemos poner ninguna caja, por lo que $f(i, p) = -\infty$. Si $i > N$, entonces no quedan cajas para considerar, por lo que $f(i, p) = 0$. En ambos casos la invariante se cumple.

Caso inductivo: Si $i \leq N$, tenemos dos opciones:

Correctitud (II)

Caso base: Si $p < 0$, entonces no podemos poner ninguna caja, por lo que $f(i, p) = -\infty$. Si $i > N$, entonces no quedan cajas para considerar, por lo que $f(i, p) = 0$. En ambos casos la invariante se cumple.

Caso inductivo: Si $i \leq N$, tenemos dos opciones:

- No poner la caja i . En este caso, la máxima cantidad de cajas que puedo apilar es $f(i + 1, p)$, que por hipótesis inductiva es correcta.
- Poner la caja i . En este caso, la máxima cantidad de cajas que puedo apilar es $1 + f(i + 1, \min(p - w_i, s_i))$, que por hipótesis inductiva es correcta.

Correctitud (II)

Caso base: Si $p < 0$, entonces no podemos poner ninguna caja, por lo que $f(i, p) = -\infty$. Si $i > N$, entonces no quedan cajas para considerar, por lo que $f(i, p) = 0$. En ambos casos la invariante se cumple.

Caso inductivo: Si $i \leq N$, tenemos dos opciones:

- No poner la caja i . En este caso, la máxima cantidad de cajas que puedo apilar es $f(i + 1, p)$, que por hipótesis inductiva es correcta.
- Poner la caja i . En este caso, la máxima cantidad de cajas que puedo apilar es $1 + f(i + 1, \min(p - w_i, s_i))$, que por hipótesis inductiva es correcta.

Por lo tanto, $f(i, p)$ devuelve la máxima cantidad de cajas que puedo apilar usando sólo las cajas $i, i + 1, \dots, N$ y teniendo p de peso disponible para poner arriba de la última caja puesta.

Posible PD alternativa

Otra posible función recursiva, que tiene la ventaja de ser completamente polinomial, es la siguiente:

$f(i, l)$ = máximo capacidad de soporte para una pila de longitud l formada usando solo las primeras i cajas.

Posible PD alternativa

Otra posible función recursiva, que tiene la ventaja de ser completamente polinomial, es la siguiente:

$f(i, l)$ = máximo capacidad de soporte para una pila de longitud l formada usando solo las primeras i cajas.

$$f(i, l) = \begin{cases} \infty & \text{si } l \leq 0 \\ 0 & \text{si } i = 0 \text{ y } l > 0 \\ \max \begin{cases} f(i-1, l) \\ \min(f(i-1, l-1) - w_i, s_i) \end{cases} & \text{si } i > 0 \text{ y } l > 0 \end{cases}$$

Respuesta:

Posible PD alternativa

Otra posible función recursiva, que tiene la ventaja de ser completamente polinomial, es la siguiente:

$f(i, l)$ = máximo capacidad de soporte para una pila de longitud l formada usando solo las primeras i cajas.

$$f(i, l) = \begin{cases} \infty & \text{si } l \leq 0 \\ 0 & \text{si } i = 0 \text{ y } l > 0 \\ \max \begin{cases} f(i-1, l) \\ \min(f(i-1, l-1) - w_i, s_i) \end{cases} & \text{si } i > 0 \text{ y } l > 0 \end{cases}$$

Respuesta: máximo l tal que $f(N, l) \geq 0$

Complejidad:

Posible PD alternativa

Otra posible función recursiva, que tiene la ventaja de ser completamente polinomial, es la siguiente:

$f(i, l)$ = máximo capacidad de soporte para una pila de longitud l formada usando solo las primeras i cajas.

$$f(i, l) = \begin{cases} \infty & \text{si } l \leq 0 \\ 0 & \text{si } i = 0 \text{ y } l > 0 \\ \max \begin{cases} f(i-1, l) \\ \min(f(i-1, l-1) - w_i, s_i) \end{cases} & \text{si } i > 0 \text{ y } l > 0 \end{cases}$$

Respuesta: máximo l tal que $f(N, l) \geq 0$

Complejidad: $O(N^2)$

Posible PD alternativa

Otra posible función recursiva, que tiene la ventaja de ser completamente polinomial, es la siguiente:

$f(i, l)$ = máximo capacidad de soporte para una pila de longitud l formada usando solo las primeras i cajas.

$$f(i, l) = \begin{cases} \infty & \text{si } l \leq 0 \\ 0 & \text{si } i = 0 \text{ y } l > 0 \\ \max \begin{cases} f(i-1, l) \\ \min(f(i-1, l-1) - w_i, s_i) \end{cases} & \text{si } i > 0 \text{ y } l > 0 \end{cases}$$

Respuesta: máximo l tal que $f(N, l) \geq 0$

Complejidad: $O(N^2)$

Spoiler: Como quienes siguen LCC verán en Complejidad Computacional, si las cajas estuvieran ponderadas no existiría una solución polinomial a este problema (es NP-completo, pensar en cómo reducir Mochila a este problema).

¿Hacemos un break?

